Depth First Search

AlgoExpert · Easy · Graphs

The problem

- Given a tree-like graph of named nodes, return names in DFS (pre-order) order.

Intuition

- Depth-first search visits a node, then fully explores each child before moving to the next sibling.

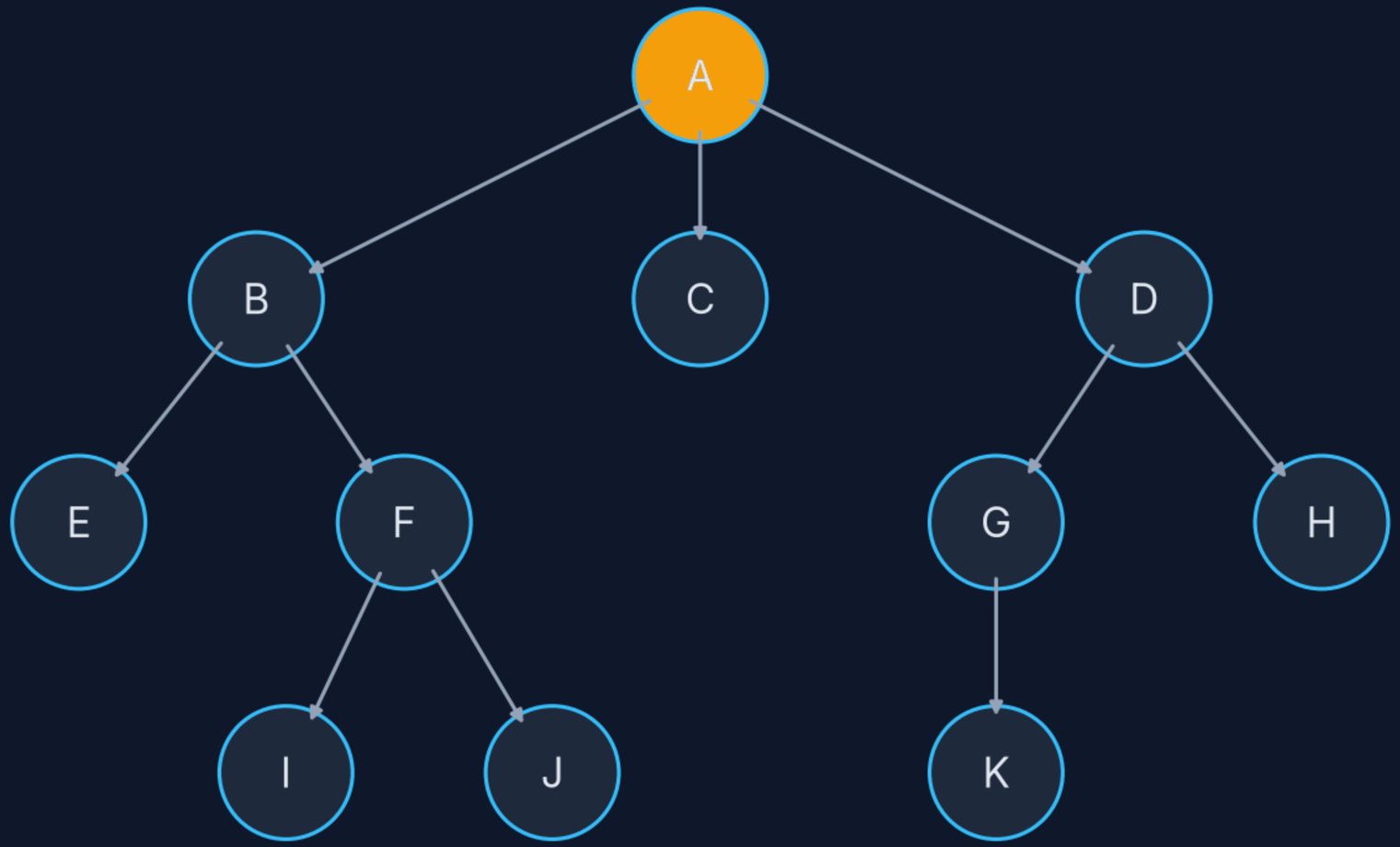
Brute force vs optimal

- DFS is already linear: O(V + E)
- Alternative: BFS gives a different (level) order
- Iterative DFS needs children pushed in reverse

Approach

- 1. Append the node's name
- 2. Recurse into each child in order
- 3. Return the collected list

Worked example

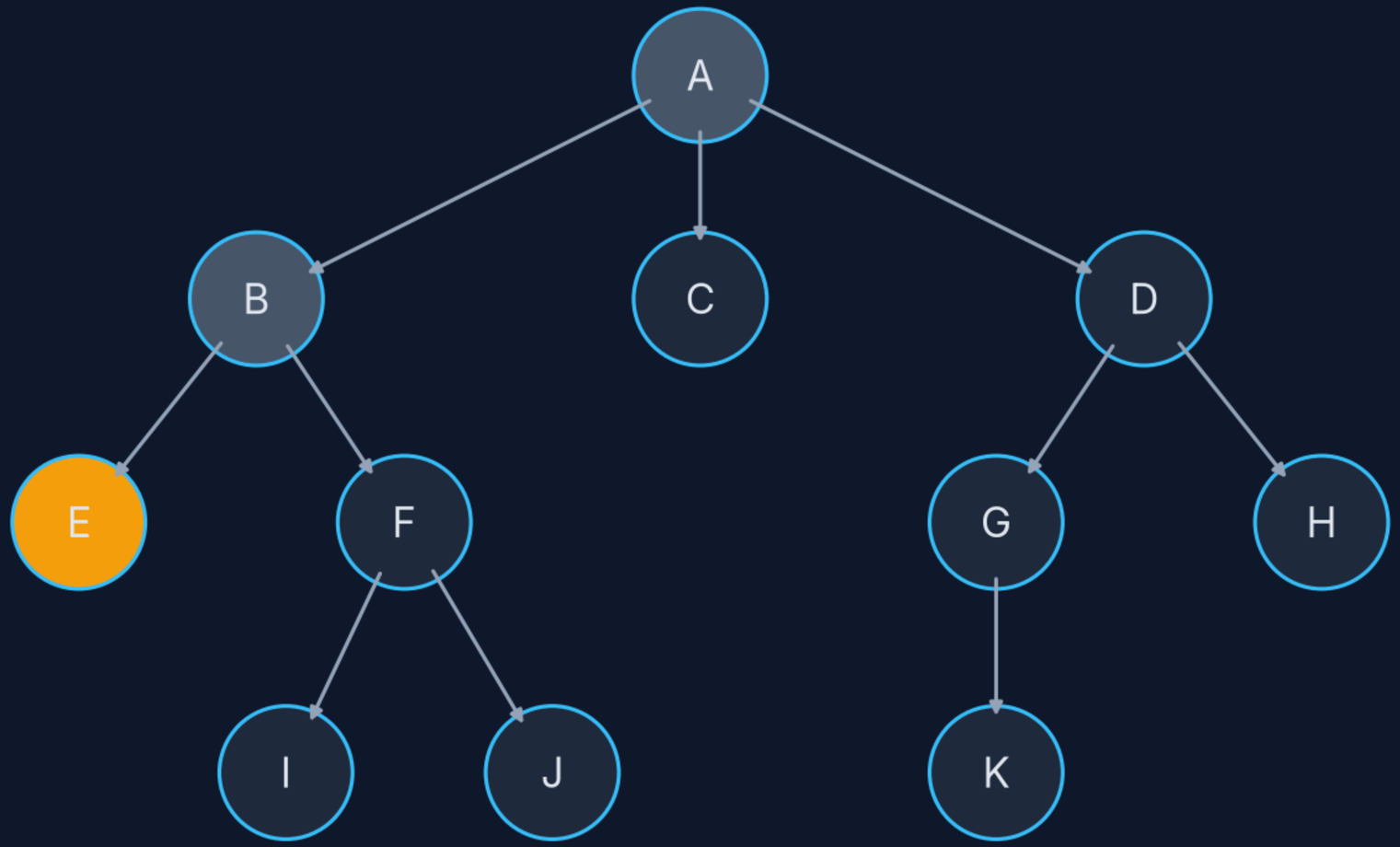



```
visited:

A
```

Visit A → order: A

Worked example

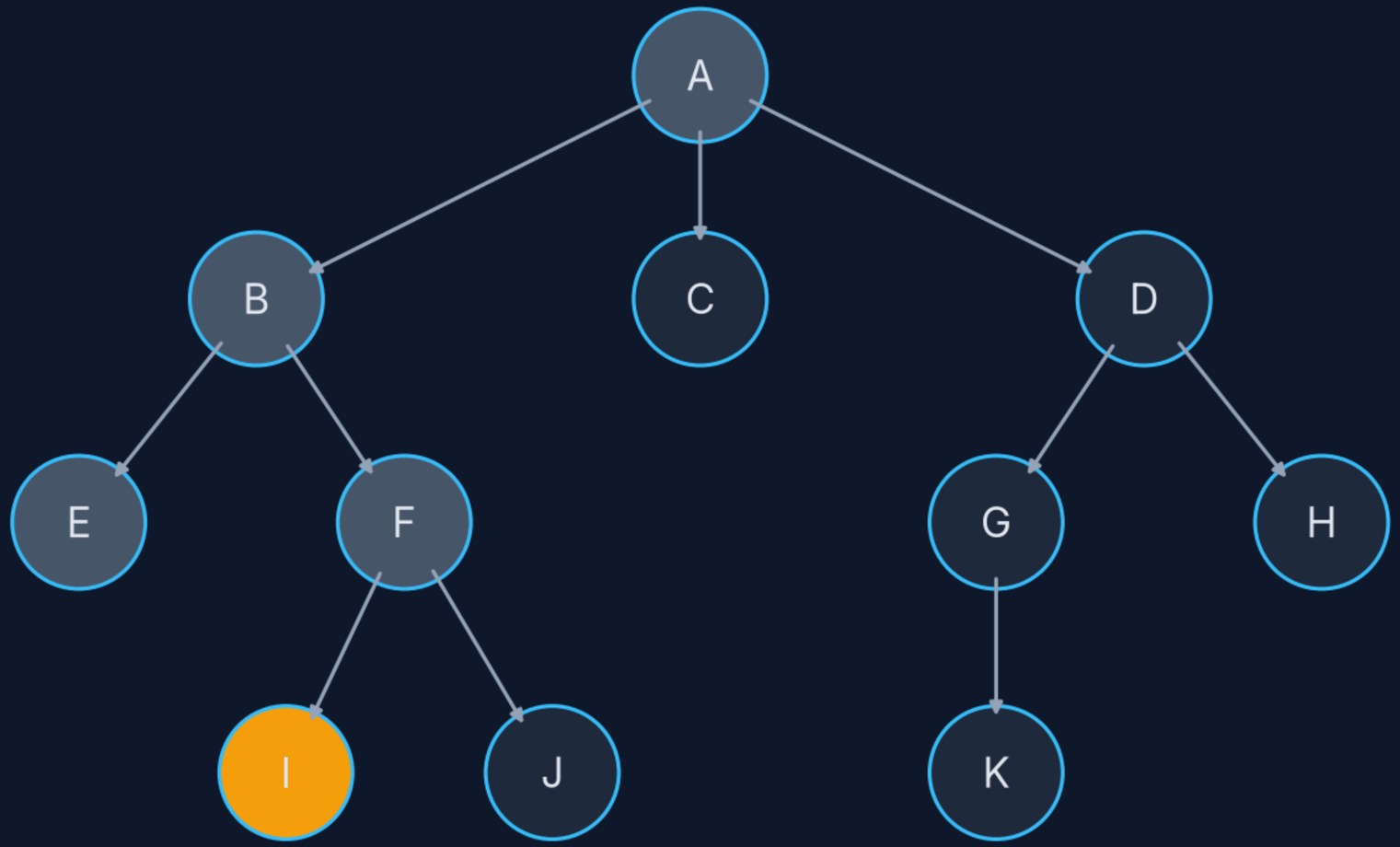



```
visited:

A B E
```

Visit E → order: A B E

Worked example

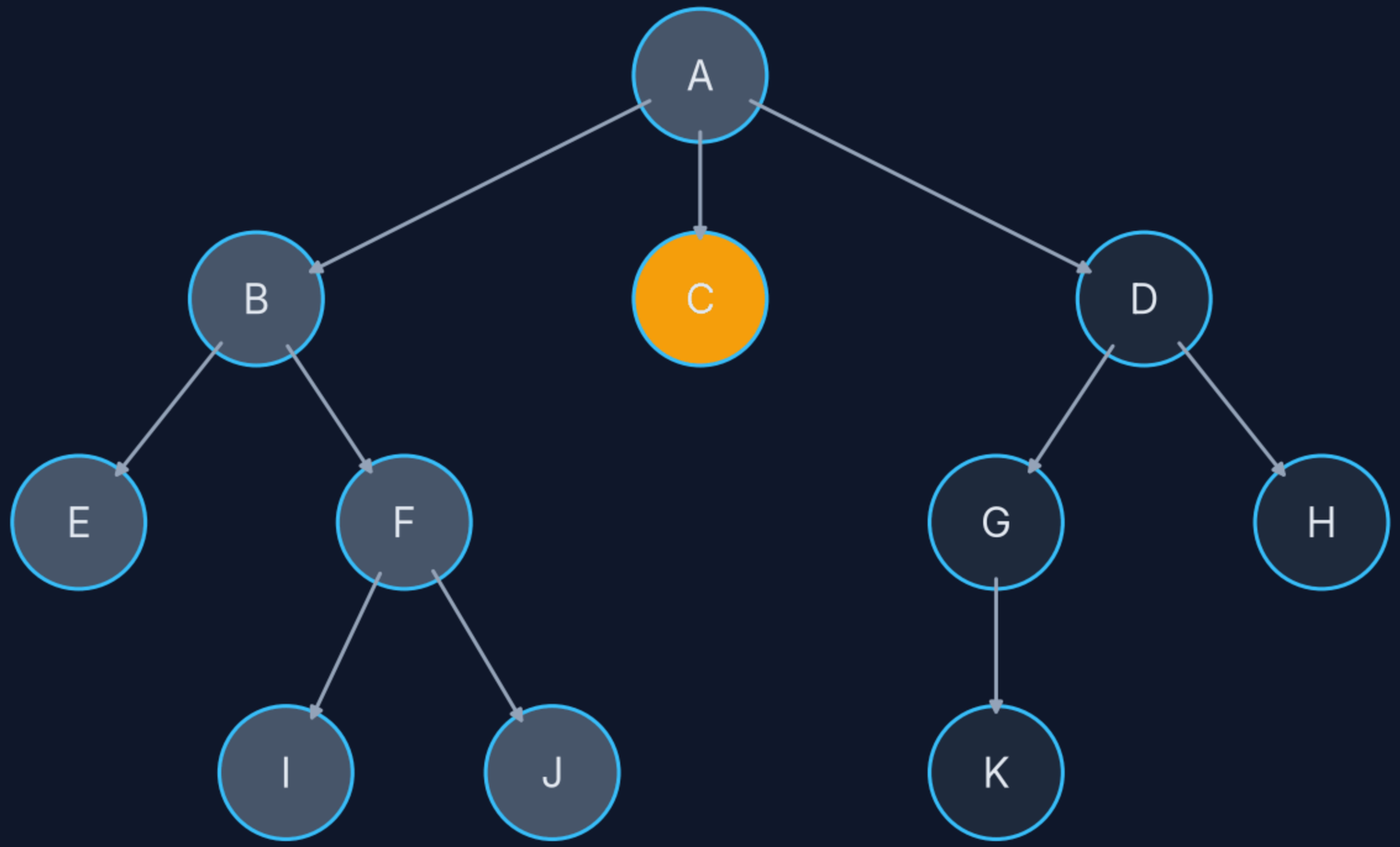



```
visited:

A B E F I
```

Visit I → order: A B E F I

Worked example

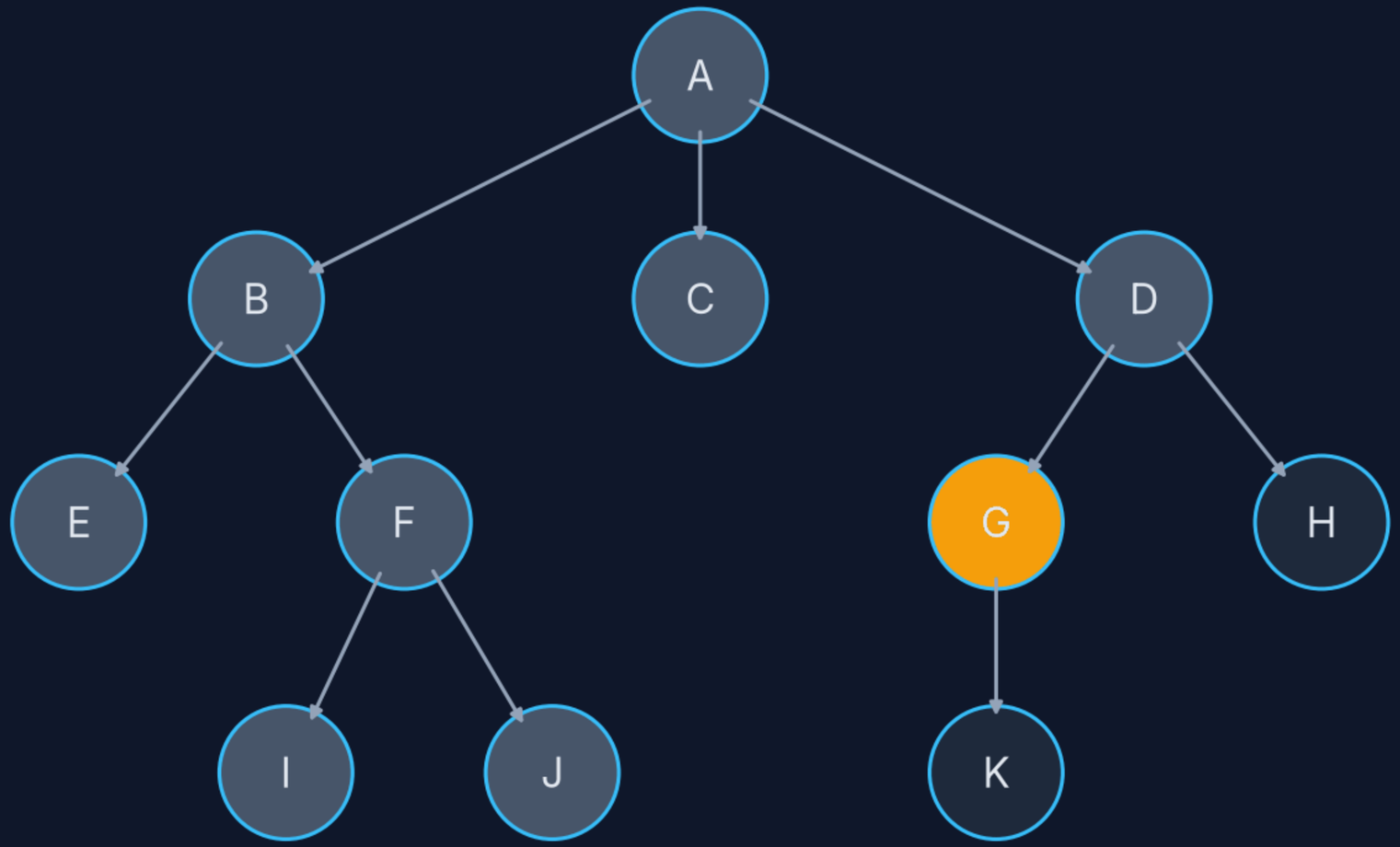



```
visited:

A B E F I J C
```

Visit C → order: A B E F I J C

Worked example

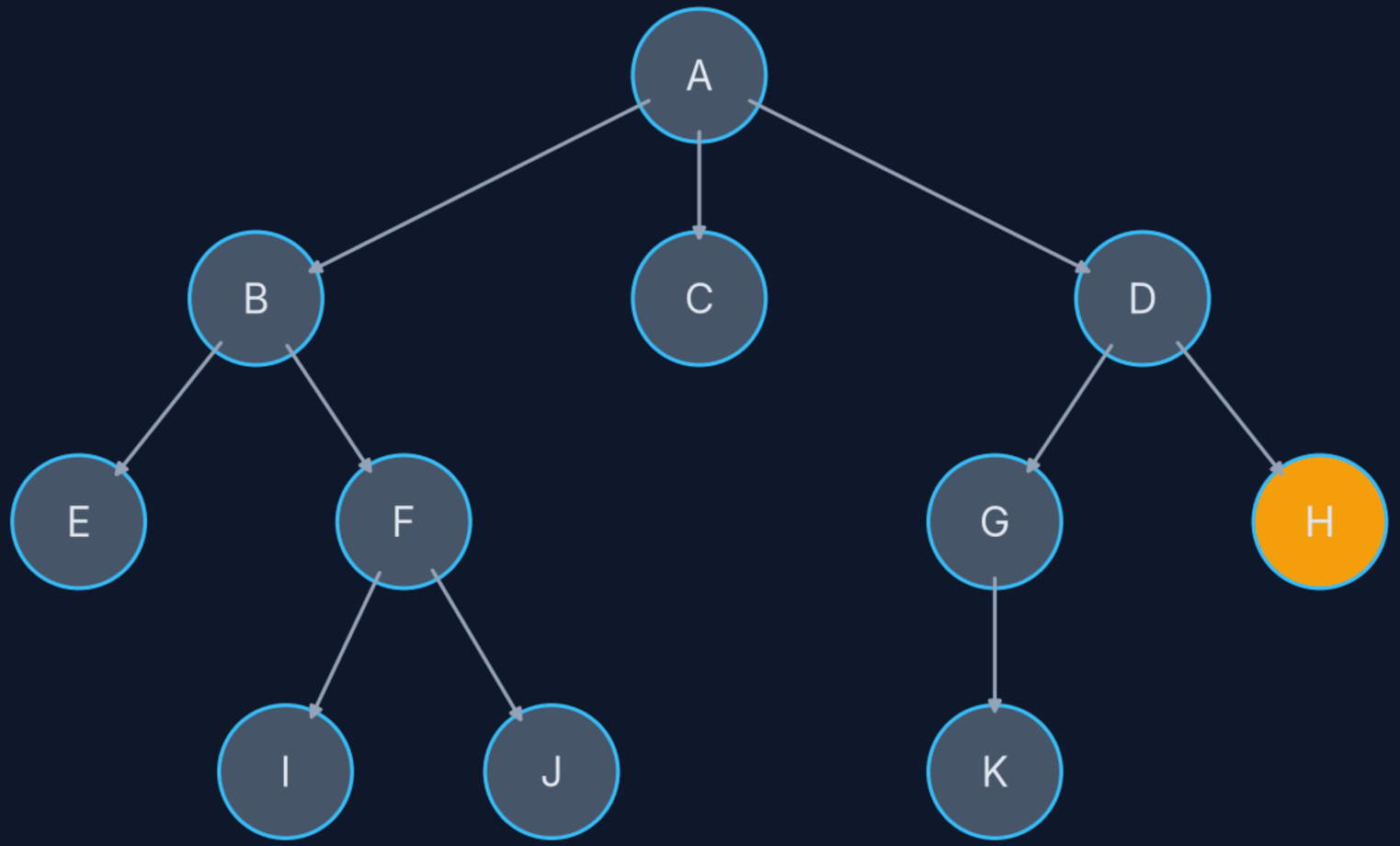



```
visited:

A B E F I J C D G
```

Visit G → order: A B E F I J C D G

Worked example




```
visited:

A B E F I J C D G K H
```

Visit H → order: A B E F I J C D G K H

Java solution

```java
public class DepthFirstSearch {
    public static class Node {
        public final String name;
        public final List<Node> children = new ArrayList<>();
        public Node(String name) { this.name = name; }
        public Node addChild(String name) { children.add(new Node(name)); return this; }
        public List<String> depthFirstSearch(List<String> out) {
            out.add(name);
            for (Node c : children) c.depthFirstSearch(out);
            return out;
        }
    }
}
```

Complexity

- Time: O(v + e)
- Space: O(v)

Edge cases & pitfalls

- Child order matters: left to right
- Iterative: push children reversed onto the stack
- General graphs need a visited set (trees don't)

Interview tips & follow-ups

- Know recursive and iterative versions
- Follow-up: detect a cycle, topological sort

Recap

- Depth-first search visits a node, then fully explores each child before moving to the next sibling.
- Complexity: O(v + e)